Teste teórico 1

Linguagem Java

- Principais características
  - Encapsulamento
  - Herança
  - Polimorfismo
  - Abstração

Estrutura básica

```
public class Exemplo {
    ⋮
    public static void main (String [] args) {
```

(args: argumentos)

* Padrão fixo

Variáveis

Strings - variável de carácteres

Int - variável numérica inteira

Float
Double - variável numérica decimal

Boolean - True ou false

Char - Apenas Carácteres

Funções básicas

(Scammer se = new Scammer) → Pedir informação ao utilizador

se.next(Double / int / String ...) → lê o "input"

se.close → é obrigatório para "fechar" os inputs

System.out.println → É um print (Ex: "O valor é:" + (variável))

↳ ("O valor é:" + variável + "..." + + "...")

* para usar scammer

↳ import java.util.Scammer

Operadores

Atribuição

↳ `int a = 1` → "a" toma valor 1

Aritméticos

↳ `double x = 2.5 * 3.75 / 4 + 100`

"*" multiplicação

"/" divisão

"%" resto da divisão

`int 4 / 3` → parte inteira da divisão

Relacionais

↳ `boolean res = (x >= y)`

" == " comparação (igual)

" <= " menor ou igual

" >= " maior ou igual

Lógicos

↳ `int age = 11 ; boolean x = (age >= 13) && (age <= 19)`

== → igual a
!= → diferente de

&& → "e" (and)
|| → "ou" (or)

Incremento e decremento

++a ou --a
↳ a variável é alterada antes de ser usada na função

a++ ou a--
↳ a variável é usada na função e só depois é alterada

Vetores

int [] vet1 ou int vet2 []
↑
mais usada

int [] v1 = new int[3]

↳ vetor com 0, 0, 0

int [] v2 = {1, 2, 3}

↳ vetor com 1, 2, 3

Aceder ao vetor

∴ int [] tabela = new int[3]

↳ de 0 a 2

Multidimensionais

```
int lista[]; // lista é um vetor de inteiros
int[] lista;  // equivalente

int[] lista={10, 20, 30, 40}; //  declaração e inicialização

int lista[] = new int[20];  // vetor de 20 inteiros

String[] texto = new String[200]; // vetor de 200 strings

int[][] tabela = new int[30][20]; // 30 linhas x 20 colunas
int tabela[][] = new int[30][20];  // idem
```

Instruções Condicionais

Instrução If

```
if (Expressão booleana)
    ...
else if
    ...
else
    ...
```

Instrução case

```
switch (expressão) {

    case valor 1:
        bloco 1
        break;

    case valor 2:
        bloco 2
        break;

    default:
        bloco final

}
```

Ciclo While

```
do
   bloco_a_executar;
while (condição);
```

– Exemplo:

```
Scanner sc = new Scanner(System.in);
int nota;
do {
   System.out.println("Insira a nota do aluno.");
   nota = sc.nextInt();
} while ( (nota > 20) || (nota < 0) );
sc.close();
```

Ciclo for

❖ Exemplo 1

```
for (int i = 1 ; i <= 10 ; i++)
   System.out.println(i+" * "+ i +" = " + i*i);
```

```
1 * 1 = 1
2 * 2 = 4
3 * 3 = 9
4 * 4 = 16
5 * 5 = 25
6 * 6 = 36
7 * 7 = 49
8 * 8 = 64
9 * 9 = 81
10 * 10 = 100
```

❖ Exemplo 2

```
int[] tb = new int[10];
for (int i = 0 ; i < tb.length ; i++)
   tb[i] = i * 2 ;
for (int i = 0 ; i < tb.length ; i++)
   System.out.print(tb[i] + ", ");
```

```
0, 2, 4, 6, 8, 10, 12, 14, 16, 18,
```

Métodos Estáticos

- Definição de classe (ficheiro ....java)
  - ↳ Public class ...
    - ↑ deve conter uma classe pública
- Deve ser declarada como public
  - main → "static void"

→ Métodos estáticos

↳ função → realiza tarefa
Tem zero ou + argumentos
retorna zero ou valor de saída

→ Aplicações

↳ funções matemáticas
construir programas modelares

Implementar

↳ Criar nome
Declarar os argumentos
especificar o valor de retorno
terminar com a instrução de retorno

Exemplo:

```
class Main {

public static void main(String[] args) {

        myFunction();

}
private static void myFunction() {

}
```

String

São objetos imutáveis (constantes)

- Aceder ao comprimento (length)
- Aceder aos caracteres charAt(int index)

Classes e Objetos

→ São especificações para criar objetos

↳ Descrevem tipos de dados de compõem o objeto
→ Métodos que o objeto pode executar

Exemplo:

```
public class Book {

    String Title;
    int pubYear;

    String getTitle() {
        return title;
        ⋮

}
```

- São implementadas dentro da package
- Nomenclatura → (ficheiro Exemplo.java)

↳ O ficheiro deve conter uma classe pública
↓
public

Objetos

↳ São instâncias de classes

→ Devem ser explicitamente criados

Ex:

- `Book oneBook = new Book();`
- `Book otherBook = new Book();`

...

Os objetos são manipulados através de referências

```
Person name1, name2;
name1 = new Person("...");
name2 = name1
```

Métodos

↳ Métodos, mensagens, funções, procedimentos

→ Notação (... . (método))

receptor → classe

→ referência para um objeto

`Math.sqrt(...)`

`otherBook.setPubYear(...)`

Todos os objetos em Java entendem a mensagem `toString()`

Inicialização de membros

↳ Dentro da classe → declaração de variáveis

↓

é igual para todos os objetos da classe

↓

CONSTRUTOR

operador new

```
Car c1 = new Car();
```

Construtor é identificado pelo nome da classe

Não retorna valor

Assume o nome da classe

Pode ter parâmetros de entrada

É chamado uma vez → na criação do objeto

Sobreposição

- Podemos usar o mesmo nome em várias funções
  - Desde tenham argumentos distintos.
    - → A ligação estática verifica a assinatura da função (nome + argumento).
    - → Não é possível distinguir funções pelo valor de retorno.

Referência this

- Pode ser utilizado dentro de cada objeto para se referenciar ao mesmo objeto.
  - → Retornar num dado método, a referência para esse objeto

Conceito static

- não tem associado a referência this
  - → não é possível invocar métodos não estáticos a partir de métodos estáticos
  - → Não são associados a objetos

Encapsulamento

↳ Separação entre aquilo que não pode mudar (interface) e o que pode mudar (implementação).

→ Controlo de visibilidade da interface (public, protected, default, private).

public - pode ser usado em qualquer classe

protected - visível dentro do mesmo package e classes derivadas

private - apenas visível dentro da classe

- Permite esconder os dados internos de um objeto
- Todos os atributos deverão ser privados.

Para aceder à parte privada deve ser efetuada sempre, através de funções da interface pública.

getters / setters → Modifica o estado do objeto

↓

devolve o valor atual de um atributo

Métodos privado

↳ são utilizados internamente por outros métodos da classe.

Relações entre Classes

Herança → RelogioDigital e RelogioAnalógico

Composição → RelogioDigital => Pilha

Herança

↳ class CDeriv extends CBase

↳ Tem acesso ao dados e métodos de CBase que não são privados

```
class Person {
        private String name;
        public Person(String n) { name = n; }
        public String name() { return name; }
        public String toString() { return "PERSON";}
}
class Student extends Person {
        private int nmec;
        public Student(String s, int n) { super(s); nmec=n; }
        public int num()   { return nmec; }
        public String toString() { return "STUDENT"; }
}
public class Test {
        public static void main(String[] args)  {
           Person p = new Person("Joaquim");
           Student stu = new Student("Andreia", 55678);
           System.out.println(p + " : " + p.name());
           System.out.println(stu + " : " + stu.name() + ", " + stu.num());
        }
}
```

Base

Derivada

PERSON : Joaquim

STUDENT : Andreia, 55678

Herança e controlo de acesso

- **public** na classe — devem ser public nas subclasses.
- **protected** na classe — devem ser public nas subclasses.
- **private** não são herdados

Final

↳ não pode ser mudado

```
final int i1 = 9;

final class Rato {
        ⋮
```

Polimorfismo

↳ O tipo declarado na variável não precisa de ser exatamente o mesmo tipo do objeto para o qual aponta — pode ser de qualquer tipo derivado.